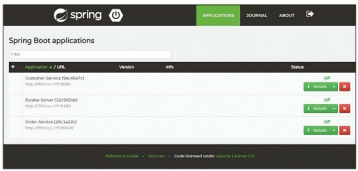

Figure 2: Admin server UI

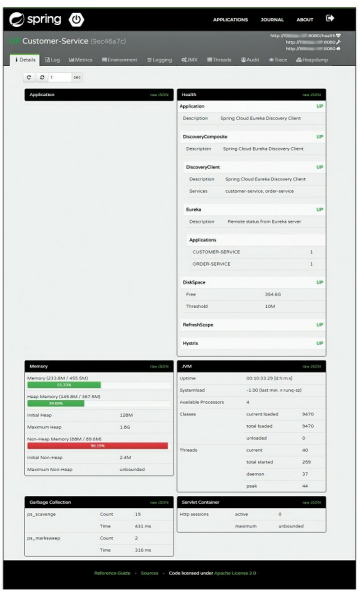

Figure 3: Detailed view of Spring Boot Admin

steps. All Spring Boot applications that we now create will act as Spring Boot Admin clients. To make the application an admin client, add the dependency given below along with the actuator dependency. In this demo, I have created three applications: Eureka Server, Customer Service and Order Service.

```
<dependency>
        <groupId>de.codecentric</groupId>
        <artifactId>spring-boot-admin-starter-client</
artifactId>
</dependency>
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</
artifactId>
</dependency>
```

Add the property given below to the *application.properties* file. This property tells us where the Spring Boot Admin Server is running. Hence, the clients will register with the server.

```
spring.boot.admin.url=http://localhost:1111
```

Now, if we start the Admin Server and other Spring Boot applications, we will be able to see all the admin clients' information in the Admin Server dashboard. As we started our Admin Server on port 1111 in this example, we can see the dashboard at *http://<host_name>:1111*. Figure 2 shows the Admin Server UI.

A detailed view of the application is given in Figure 3. In this view, we can see the tail end of the log file, the metrics, environment variables, the log configuration where we can dynamically switch the log levels at the component level, the root level or package level, and other information.

Let's now look at another feature called *notifications* from the Spring Boot admin. This notifies the administrators when the application status is DOWN or when the application status is coming UP. Spring Boot admin supports the following channels to notify the user.

- Email notifications
- Pagerduty notifications
- Hipchat notifications
- Slack notifications
- Let's Chat notifications

In this article, we will configure Slack notifications. Add the properties given below to the Spring Boot Admin Server's *application.properties* file.

```
spring.boot.admin.notify.slack.webhook-url=https://hooks.
slack.com/services/T8787879tttr/B5UM0989988L/0000990999VD1hV
t7Go1eL //Slack Webhook URL of a channel
spring.boot.admin.notify.slack.message="*#{application.
name}*  is *#{to.status}*" //Message to appear in the
channel
```

Since we are managing all the applications with the Spring Boot Admin, we need to secure its UI with a login feature. Let us enable the login feature to the Spring Boot Admin Server. I am going with basic authentication here. Add the Maven dependencies give below to the Admin